Wrapper Functions Basic Writeup

Abstract

Wrapper functions are a common class of functions that convert many lines of code into a function with a few inputs for key modifications. Wrappers can greatly simplify using complex code, making it far more accessible to less experienced users. However, many possible modifications to the original code may become inaccessible, making edits not foreseen by the wrapper creator difficult to impossible for users. There are a few ways for wrapper creators to make future edits to their function easier. These start with the function being self-contained (Do I put a definition for self-contained here? Or is it sufficient it is defined when brought up in the body of the paper?), but also include a mechanism for users to view and edit code, and potentially even save their modified versions of the wrapper function for future use.

What are Wrapper functions

Basic description

Wrapper functions are a broad category of functions that typically aim to take a more complex task and create a simple way of recreating it, often by limiting the number of required inputs (This is pretty broad, should I focus on a more narrow category of wrapper functions. I kind of bounce back and forth between ones that generate figures and ones that generate models). To accomplish this, a more complex block of code is wrapped up in a function and many inputs have values selected by the creator. A few key inputs are left outward-facing to be adjusted by the user with the wrapper function. However, many inputs may be permanently unadjustable by the user. Common uses for wrapper functions include complex graphics where there may be hundreds of lines of code, but only a couple are typically of interest to the user. Another common use is in modeling functions where a single function call may test a great number of models, assumptions about the data, clean the data, and return what it considers to be the best-fitting model with very little user input. (Are there other major uses I'm blanking on)

Popular examples

A popular class of wrapper functions helps users create complex ggplots involving multiple layers with as little as one line of code (Wickham 2016). A major example of this style is `autoplot()`, which uses the class of the object given to it to choose an appropriate ggplot display. This allows users to quickly plot objects like time series, model objects, or dimension reduction results without needing to know all of the individual ggplot layers being created. The **survminer** package provides a more domain-specific example with `ggsurvplot()`, a wrapper that builds survival curves from fitted survival objects and can add common survival-analysis features such as confidence intervals, censoring marks, p-values, and risk tables (Kassambara et al. 2026). The **ggpubr** package takes a broader plotting-helper approach, providing functions such as `ggboxplot()` and `ggscatter()` that wrap common ggplot patterns behind simple arguments for labels, grouping, colors, and statistical annotations (Kassambara 2026). There are also packages created as companions for textbooks that often have wrapper functions to help create graphics like those seen in the book. The textbook **Time Series Analysis and Its Applications** (Shumway and Stoffer 2025) has its own package **astsa** with several wrappers for creating similar graphics, and for quickly determining models (Stoffer 2025). Even packages not focused on producing graphics may have one or two wrappers for creating specific visuals related to the goal of their package.

Another important class of wrapper functions is involved in modeling. The **fable** package in time series has modeling functions that can take in as little as just a dataset and from there the function will make numerous decisions for the user to return what it determines to be the best model for their data (O'Hara-Wild et al. 2024). (Do I provide more modeling examples, do i provide more details.)

Advantages

The clear advantage to creating a wrapper function is enabling users to create complex figures or well-fitted models with ease. For a user looking to complete a single application with as little friction as possible this is ideal. With a wrapper function the creator can enable beginner users to recreate their more advanced figures or models. This may be important if the user has colleagues that need to replicate work in a similar style, since it reduces the expertise needed by the user, and increases consistency. If none of the inputs have been fixed and made inaccessible to the user, then wrapper functions offer primarily upside. In some cases what would have been hundreds of lines of code becomes one.

Outside of mere ease of use, wrapper functions can allow a user to lean on the experience of the creator of the function. If the creator is an expert in the area, their preselected values for inputs, or methods for automatically determining them, may be more reasonable than anything an inexperienced user could guess. The user is less likely to violate unknown assumptions if the function checks those itself. Core code may have even been converted into C or other

faster running languages, making the function faster than anything the user could easily write themselves.

Issues

Issues can begin to crop up if an input that was fixed by the creator is something you need to adjust. It could be as simple as switching the line colors of a plot to be more colorblind friendly. Depending on how the wrapper function was written, making minor adjustments can range from a slight inconvenience to a total nightmare as you try to navigate the depths of their package architecture searching for where the color of those lines was actually encoded. The simplest wrapper function is just a block of code that has been placed in a function. We call these styles of wrapper functions "self-contained" since the function itself contains everything it needs to function. Self-contained wrappers are very easy to adjust since you can take the block of code and create a new wrapper with the code slightly adjusted, or by just adding a new input to allow yourself to adjust what was previously fixed. Difficulties arise when the function is not self-contained. The wrapper may call many other functions, each of which could conceivably contain the code you need to adjust, requiring you to search through all of them. Outside of just calling other functions, the function could use S3, S4, or R6, giving it custom methods that could further adjust how the output looks. If the function calls compiled C code that contains what you want to adjust, there may be nothing you can do outside of writing it all yourself or finding a new package.

Beyond difficulty with modifying the function, wrapper functions can be very difficult to bug fix. The resulting error when something goes wrong can be very unclear, particularly if the error is in another function called by the wrapper function. The user often struggles to determine if the issue is with their inputs, or if they are using the wrapper function incorrectly. The error may be fatally incomprehensible if the error is with a compiled C function.

Finally, if a user would like to use the output of the wrapper as part of some greater project, they may be unsure what they have actually done. (There's plenty to be said about the user not really understanding their code and what they have really done, but is that something worth going down. My proposals in this paper don't really do anything to fix that issue.)

Ways to combat Basic Issues

Package creators typically create wrapper functions to make their code as usable for users as possible. So, what steps exist to allow wrapper functions to be as simple and functional, but open to adjustment as possible? One unfortunate drawback to all the following methods is that they work better the more self-contained the function is.

Original method

One adjustment to allow users to make simple adjustments to a function could be to have an argument that by default runs the wrapper as usual, but could be switched to print out the code to be run in a new document instead of running the function. Adjustments to the code could be made live before running it. This would be helpful for very minor changes like switching the color of a line in a plot, or changing a couple fixed numeric values. While this method is easy to implement and nice for single adjustments, it is inconvenient if a user would like to consistently make the same change when using the function a number of times.

Binding method

While the method outlined in section a is nice for minor one-off changes, it would be rather inconvenient if an adjustment was desired to be used regularly. This is especially true if you want to use the wrapper function many times, all with the same adjustment. To build off the previous method, you could instead bind the adjusted code to a new object in the working environment. The wrapper function could be adjusted to take in modified code and run it instead of its original. The user could then easily make their adjustment one time and continue to use their modified version of the original wrapper function. The wrapper function could even allow the name of the binding to be passed into the function itself and then assign it to the global environment after adjustments have been made. This creates a more semi-permanent solution, but the edited code living in the global environment requires the user to re-create it, or save it as an RDS, between sessions. It also requires the creator to add several arguments to every wrapper function they would like to be editable by the user.

Template method

The previous methods have both involved editing wrapper functions to have their typical functions as well as arguments that give them editing functions. For a single wrapper this can be useful, but for a package containing multiple it may be cleaner to have a single editing function that works for all wrapper functions. In this method the code bodies of wrapper functions are stored in the package, and a dedicated templating function calls them into a new document. An argument for the template's name is given, and when edits to the function are complete the edited code is saved as an RDS, as opposed to just being in the global environment. These templates allow for long-term use of modified code for the user, and even for templates to be shared between users if several would like to make the same adjustment. The only adjustment that needs to be made to the wrapper functions themselves in this method is allowing for a template to be provided and running the template code instead.

How to Implement

To give a running example of how this might work, we've created a wrapper function called raincloud that takes in a tibble with two variables, value and class. The default wrapper produces a violin plot with jittered observations, a narrow boxplot, and a median point.

```
set.seed(17)
a <- tibble::tibble(
  value = c(rnorm(2e2, 1), 4 * rbeta(2e2, 1, 4)),
  class = c(rep("a", 2e2), rep("b", 2e2))
)
```

```
raincloud_original(x = a)
```

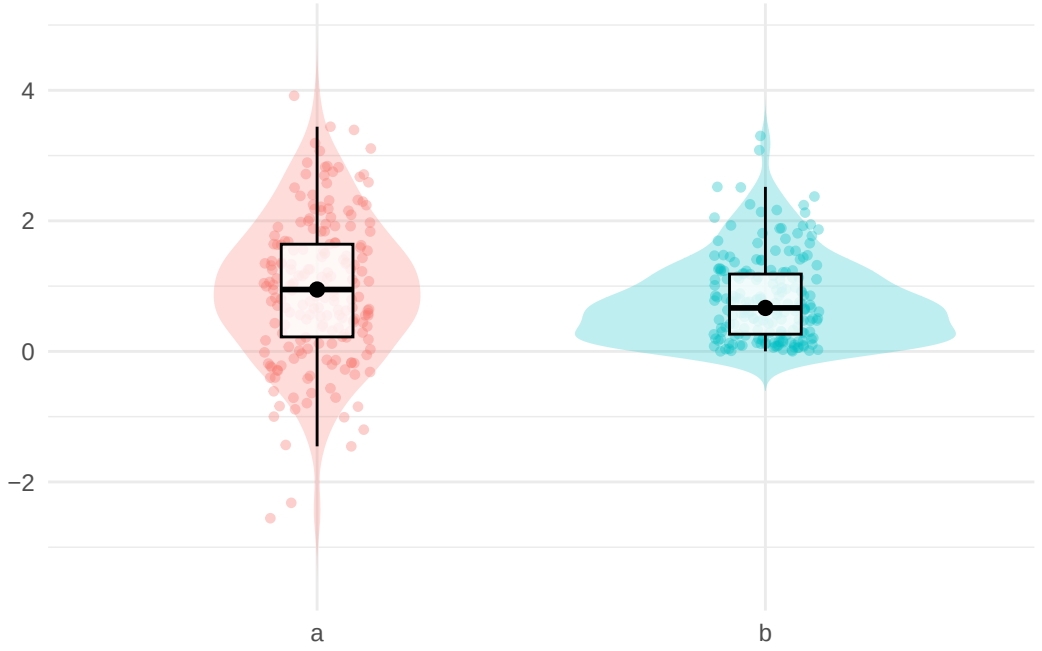

Figure 1: Default raincloud wrapper output.

Original method

The original method is intended for a one-off change. The user first runs the plot in its default form, but finds that it's hard to see the points with the boxplot on top of them. To make an adjustment they can rerun the function with format set to `"edit"`.

```
raincloud_original(x = a)
raincloud_original(x = a, format = "edit")
```

Running `raincloud_original(x = a, format = "edit")` opens an unnamed R script containing the internal plotting code wrapped as editable code. The script begins with a short message describing how to edit and then assigns the code to `code_output$code`. To make the points clearer, the user swaps the order of the jitter and boxplot layers.

```
code_output$code <- expr({print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
    geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
    geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
    geom_jitter(width = 0.12, alpha = 0.35, size = 1.2)+
    stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
    labs(x = NULL, y = NULL)+
    theme_minimal()+
    theme(legend.position = "none")
})
```

After making the change, the user runs the edited block once to save the revised code object. Then they call `raincloud_adjust()` to evaluate the modified version in the original function environment.

```
raincloud_adjust()
```

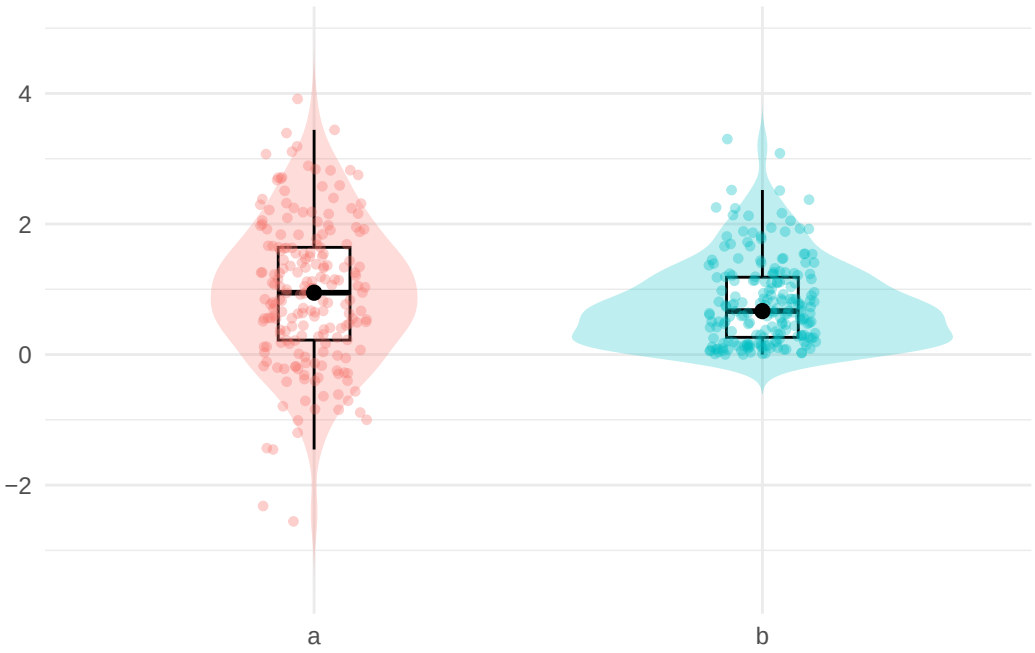

Figure 2: Original-method edit: the boxplot is drawn behind the jittered points.

This method is simple, but the modified code lives in the default `code_output` object. If the user wants several different edited versions, or wants to name a particular version, the next method is more convenient.

Binding method

The user finds that points are a little clearer now that they are on top, but that the color blends into the violin plot background. They decide they would like to change the shape of the points and make them black so they stand out better. In case they want to make further adjustments to the code, they use the binding method and assign it to `black_points` in the local environment.

```
raincloud_binding(x = a)
black_points <- raincloud_binding(x = a, format = "edit")
```

The editable script again contains the plot code, but the wrapper returns an object with class `bnd`. The user fills in the object name on the left side of `$code`, edits the plot code, and runs the block.

```
black_points$code <- expr({print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
```

```
    geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
    geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
    geom_jitter(width = 0.12, alpha = 0.35, shape = 5, color = "black")+
    stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
    labs(x = NULL, y = NULL)+
    theme_minimal()+
    theme(legend.position = "none")
})
```

Because `black_points` has a print method, the user can display the modified plot by evaluating the object itself.

```
black_points
```

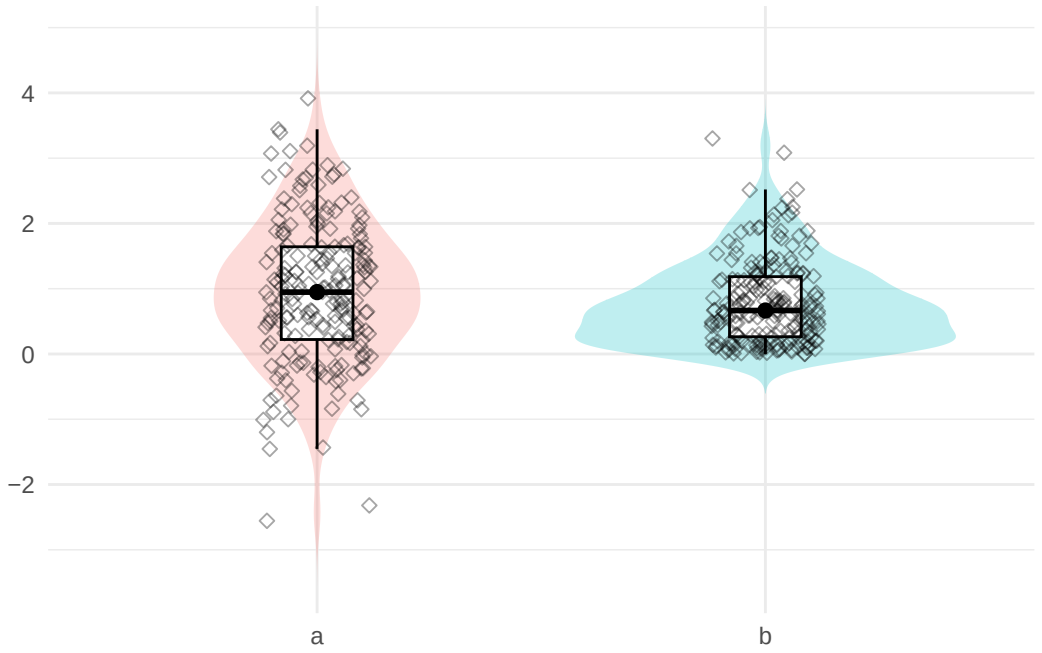

Figure 3: Binding-method edit: jittered points are drawn on top as black diamonds.

The adjusted version now has a name in the user's workspace, so they won't have to worry about overwriting it. The limitation is that it still lives inside the current R session unless the user saves it separately.

Template method

The template method makes the edited version more durable. Instead of storing the changed code only in the current environment, the user creates an editable `.R` template file:

```
make_template("raincloud", "clearer_points")
```

This copies the package's raincloud plotting template into `clearer_points.R` and opens it for editing. The user can then apply the previous two changes and make one additional adjustment: increasing the jitter width from `0.12` to `0.35`, giving the diamond points more horizontal room to spread out.

```
print(
  ggplot(x, aes(x = class, y = value, fill = class, color = class))+
    geom_violin(alpha = 0.25, trim = FALSE, width = 0.85, color = NA)+
    geom_boxplot(width = 0.16, outlier.shape = NA, fill = "white", color = "black")+
    geom_jitter(width = 0.35, alpha = 0.35, shape = 5, color = "black")+
    stat_summary(fun = median, geom = "point", size = 2.2, color = "black")+
    labs(x = NULL, y = NULL)+
    theme_minimal()+
    theme(legend.position = "none")
)
```

Once the file is saved, the wrapper can use the modified template directly:

```
raincloud_template(a, "clearer_points.R")
```

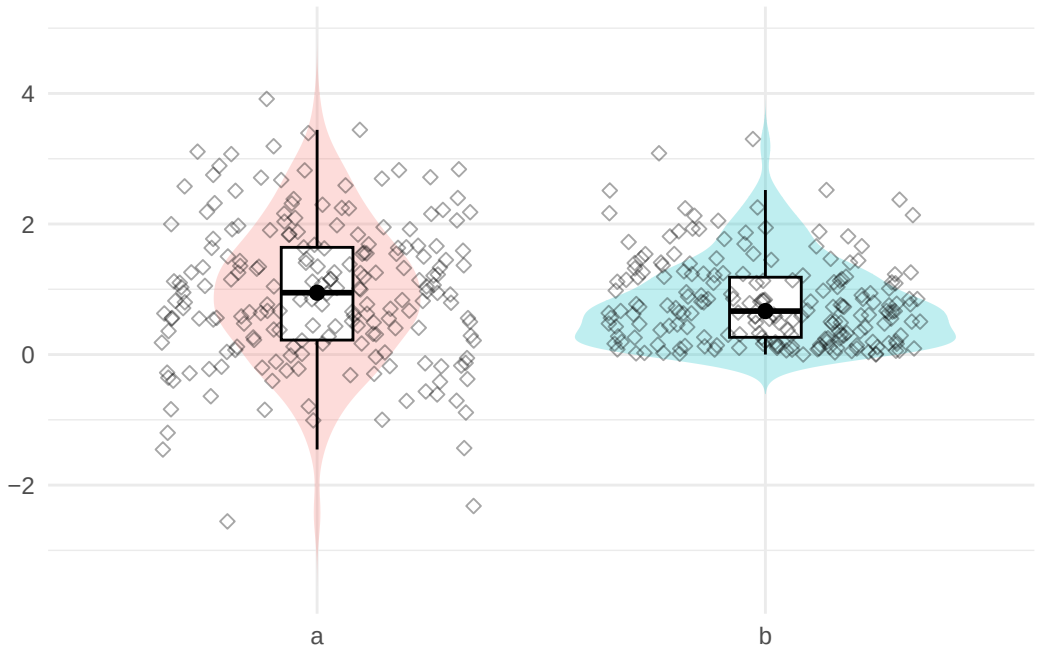

Figure 4: Template-method edit: previous edits plus wider jitter spacing.

This final version is the easiest to reuse across sessions or share with another user.

Conclusion

Considering the pros and cons of the methods described, we would have to recommend the template method as being the most valuable for users. The ability to not just adjust a function, but save those adjustments in an easily reusable and shareable way, is valuable in any collaborative setting where wrapper functions are being used. One can easily imagine creating plot templates with colors adjusted to match a company or school theme and sharing them with colleagues. The difficulty of implementation is also not particularly higher than other simpler methods described. The largest barrier to entry for all methods described remains how self-contained a wrapper function is able to be, since code can only be edited if it is accessible to users.

References

Kassambara, Alboukadel. 2026. *Ggpubr: 'Ggplot2' Based Publication Ready Plots*. https://doi.org/10.32614/CRAN.package.ggpubr.

Kassambara, Alboukadel, Marcin Kosinski, and Przemyslaw Biecek. 2026. *Survminer: Drawing Survival Curves Using 'Ggplot2'*. https://doi.org/10.32614/CRAN.package.survminer.

O'Hara-Wild, Mitchell, Rob Hyndman, and Earo Wang. 2024. *Fable: Forecasting Models for Tidy Time Series*. https://doi.org/10.32614/CRAN.package.fable.

Shumway, Robert H., and David S. Stoffer. 2025. *Time Series Analysis and Its Applications: With r Examples*. 5th ed. Springer Texts in Statistics. Springer Cham. https://doi.org/10.1007/978-3-031-70584-7.

Stoffer, David. 2025. *Astsa: Applied Statistical Time Series Analysis*. https://doi.org/10.32614/CRAN.package.astsa.

Wickham, Hadley. 2016. *Ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org.